

Human Development Index (HDI) Prediction System using Machine Learning and Flask

ProjectDescription:

The **HDI Prediction System** is a machine learning-based web application developed to predict the **Human Development Index (HDI)** using various socio-economic indicators. The project aims to assist researchers, students, and policymakers in estimating HDI values based on important development parameters. The system applies data preprocessing, exploratory data analysis, and machine learning techniques to generate accurate predictions through a simple and interactive web interface. The project follows a complete machine learning workflow, including data collection, preprocessing, feature analysis, model training, evaluation, deployment, and testing. A Flask-based web application is integrated with the trained model, enabling users to enter input values and instantly receive HDI predictions.

Problem Statement

The **Human Development Index (HDI)** is a key indicator used to measure the overall development of a country based on factors such as **life expectancy, education, and income**. Traditional methods of analyzing and estimating HDI involve manual processing of large volumes of socio-economic data, which is time-consuming, complex, and prone to errors. This makes it difficult for policymakers, researchers, and organizations to obtain quick and accurate insights for effective decision-making.

The **HDI Prediction System** addresses this challenge by leveraging **Machine Learning** to predict HDI values based on selected socio-economic indicators. The system automates the prediction process, improves accuracy, reduces analysis time, and provides a simple web-based interface for users. By enabling fast and reliable HDI predictions, the application supports data-driven policy planning, development analysis, and research.

Project Objectives

The main objectives of the HDI Prediction System are:

- Develop a machine learning model for HDI prediction.
- Analyze socio-economic data using Exploratory Data Analysis (EDA).
- Build a responsive and user-friendly web application.
- Integrate the trained model with the Flask backend.
- Generate accurate HDI predictions based on user inputs.
- Demonstrate the practical application of machine learning in socio-economic analysis.

Project Scope

The **HDI Prediction System** is designed to assist users in estimating the **Human Development Index (HDI)** based on important socio-economic indicators. The system uses a Machine Learning model to analyze development-related parameters and generate accurate HDI predictions through a simple and user-friendly web application.

The project covers:

- HDI dataset collection and analysis
- Data preprocessing and feature engineering
- Exploratory Data Analysis (EDA)
- Machine Learning model development and evaluation
- Flask-based web application development
- Model integration with the web application
- User-friendly prediction interface
- Local deployment and application testing

The current system predicts the Human Development Index based on the provided input parameters but does not include real-time data integration, cloud deployment, or advanced predictive analytics.

Scenarios

Scenario 1 – Government Planning

Government officials can use the application to estimate HDI values based on socio-economic indicators. The predicted results help analyze development levels and support evidence-based policy planning.

Scenario 2 – Researchers and Data Analysts

Researchers can evaluate the impact of different socio-economic factors on HDI by entering various input values. This enables comparative analysis and supports academic research.

Scenario 3 – Educational Institutions

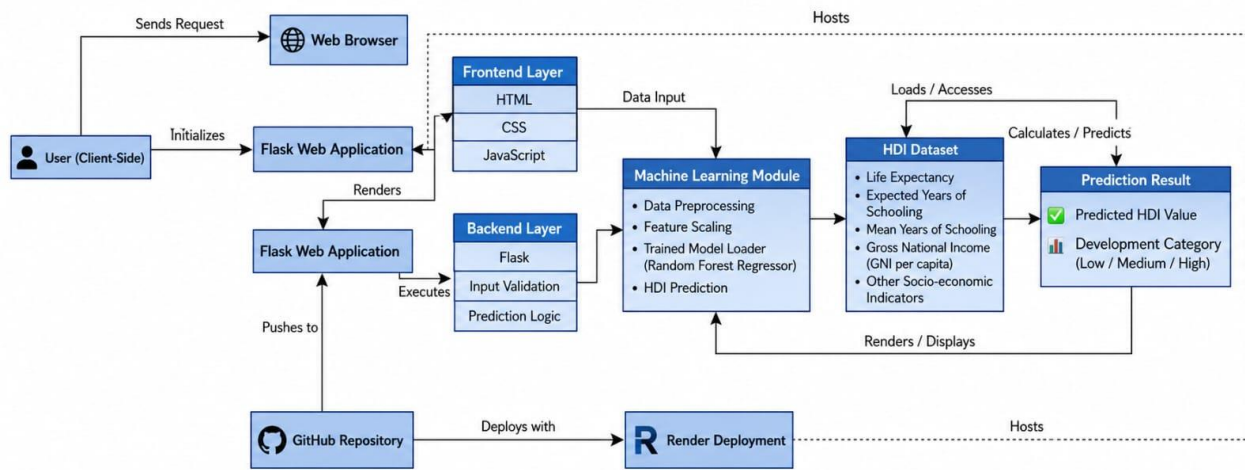
Students and educators can use the system as a learning tool to understand how Machine Learning techniques are applied in predicting the Human Development Index using real-world data.

Scenario 4 – Development Organizations

Non-governmental organizations (NGOs) and development agencies can use the application to analyze development indicators and identify regions that may require additional attention for improving human development outcomes.

This follows the same style and structure as the SmartBridge sample while being specific to your **HDI Prediction System**.

Technical Architecture:



Technical Architecture Description

The technical architecture of the **HDI Prediction System** illustrates the complete workflow of the application, showing how different components interact to predict the **Human Development Index (HDI)** based on socio-economic indicators. The system is developed using a **Flask-based web application** integrated with a **Machine Learning model** to provide accurate and efficient HDI predictions.

The process begins when a **user** accesses the application through a web browser. The **Frontend**, developed using **HTML, CSS, and JavaScript**, provides an intuitive interface where users enter the required input parameters such as **Life Expectancy, Expected Years of Schooling, Mean Years of Schooling, Gross National Income (GNI) per Capita, and other socio-economic indicators**.

The input data is sent to the **Flask Backend**, which validates the user inputs, performs necessary preprocessing, and forwards the processed data to the trained **Random Forest Regression** model. During preprocessing, feature scaling and data transformation are applied to ensure consistency with the training dataset.

The trained machine learning model loads the saved **model.pkl** file and predicts the corresponding **HDI value**. Based on the predicted value, the application also determines the **development category** (Low, Medium, High, or Very High Human Development).

Finally, the prediction results are displayed on the web interface, allowing users to quickly understand the development status based on the provided indicators. The complete application is managed using **GitHub** for version control and can be deployed using **Render** or any cloud hosting platform for public accessibility.

This technical architecture demonstrates the seamless integration of the **Frontend, Flask Backend, Machine Learning Model, and Deployment Platform**, providing a reliable, scalable, and user-friendly solution for HDI prediction.

Pre-requisites

The following software, tools, and technologies are required to develop and run the **HDI Prediction System** successfully.

Hardware Requirements

- Processor: Intel Core i3 or above
- RAM: 4 GB minimum (8 GB recommended)
- Storage: 10 GB free disk space
- Internet connection for downloading dependencies

Software Requirements

- Operating System: Windows 10/11, Linux, or macOS
- Python 3.10 or above
- Visual Studio Code (VS Code)
- Git and GitHub
- Web Browser (Google Chrome, Microsoft Edge, or Firefox)

Python Libraries

- Flask
- Pandas
- NumPy
- Scikit-learn
- Joblib
- Matplotlib
- Seaborn

Dataset Requirements

The system requires an **HDI dataset** containing socio-economic indicators such as:

- Life Expectancy
- Expected Years of Schooling
- Mean Years of Schooling
- Gross National Income (GNI) per Capita
- Historical HDI Values

Technical Knowledge

- Basic Python Programming
- Machine Learning concepts
- Data Preprocessing and Feature Engineering
- Flask Web Framework
- HTML, CSS, and JavaScript
- Git and GitHub version control

Development Environment

- IDE: Visual Studio Code
- Backend Framework: Flask
- Frontend: HTML, CSS, JavaScript
- Machine Learning Framework: Scikit-learn
- Model Serialization: Joblib (.pkl)
- Version Control: Git & GitHub
- Deployment Platform: Render

These prerequisites provide the necessary environment for building, training, deploying, and running the **HDI Prediction System** efficiently.

Project Workflow

Project Milestone

The development of the HDI Prediction System was completed in five major milestones. Each milestone focused on a specific phase of machine learning model development, web application implementation, testing, and deployment. The project followed a structured workflow to ensure accurate HDI prediction and a user-friendly application

Milestone 1: Project Planning & Environment Setup

The first milestone focused on establishing the foundation for the HDI Prediction System. During this phase, the project objectives and problem statement were defined, and an appropriate HDI dataset containing socio-economic indicators was collected. The required technologies, including Python, Flask, HTML, CSS, JavaScript, Scikit-learn, and GitHub, were selected. The development environment was configured by installing the required software, creating a virtual environment, and organizing the project into a modular folder structure. Completing this milestone ensured a well-prepared environment for the remaining stages of development.

Milestone 2: Data Preprocessing & Model Development

The second milestone focused on preparing the dataset and building the machine learning model. The collected HDI dataset was cleaned by handling missing values, removing duplicate records, and selecting relevant features. Exploratory Data Analysis (EDA) was performed to understand the relationships among socio-economic indicators. The dataset was then split into training and testing sets, and multiple machine learning algorithms were evaluated. The **Random Forest Regression** model was selected based on its prediction accuracy. The trained model and preprocessing objects were saved using Joblib for integration with the web application.

Milestone 3: Flask Web Application Development

The third milestone focused on integrating the trained machine learning model into a web application. A responsive frontend was developed using **HTML, CSS, and JavaScript**, allowing users to enter socio-economic parameters easily. The backend was implemented using **Flask**, which validates user input, preprocesses the data, loads the trained model, and generates HDI predictions. The application displays both the predicted HDI value and its corresponding development category through a simple and interactive interface.

Milestone 4: Testing & Performance Evaluation

The fourth milestone focused on validating the complete application. Different combinations of socio-economic input values were tested to verify the accuracy and consistency of predictions. Model performance was evaluated using regression metrics such as **R² Score, MAE, MSE, and RMSE**. Input validation, application responsiveness, and user interface functionality were also tested. Any identified issues were corrected to improve reliability and usability before deployment.

Milestone 5: Deployment & Documentation

The final milestone focused on deploying the HDI Prediction System and preparing project documentation. The complete source code was uploaded to **GitHub**, and the application was deployed using **Render** for public accessibility. A comprehensive README file, project report, screenshots, testing results, and technical documentation were prepared. This milestone marked the successful completion of the project and ensured that the application was ready for demonstration, evaluation, and future enhancements.

Milestone 6: Conclusion & Future Enhancements

The HDI Prediction System was successfully designed, developed, tested, and integrated using Python, Flask, and a Linear Regression model. The application accurately predicts the Human Development Index based on socio-economic indicators through a simple and interactive web

interface. The project demonstrates the practical application of machine learning in predicting development indicators and provides a foundation for further enhancements.

Milestone 1: Project Planning & Environment Setup

1.1 Introduction

The first milestone focused on understanding the project requirements and preparing the development environment for the **HDI Prediction System**. Before building the machine learning model, it was necessary to identify the problem, define clear objectives, collect an appropriate dataset, and select suitable technologies. The development environment was then configured by installing the required software, libraries, and development tools. This milestone established a strong foundation for the successful implementation of the HDI Prediction System.

1.2 Problem Identification

The Human Development Index (HDI) is a key indicator used to measure the overall development of a country based on factors such as life expectancy, education, and income. Traditional methods of analyzing HDI involve manual calculations and extensive data analysis, which can be time-consuming and prone to errors. To overcome these limitations, an intelligent HDI Prediction System was proposed. The objective is to develop a machine learning-based web application capable of predicting the HDI value using socio-economic indicators, thereby supporting researchers, policymakers, and students in analyzing development trends efficiently.

1.3 Project Objectives

The objectives identified during this milestone are:

- Develop a Machine Learning model for HDI prediction.
- Analyze socio-economic indicators affecting HDI.
- Perform data preprocessing and feature engineering.
- Compare different Machine Learning algorithms.
- Select the best-performing prediction model.
- Develop an interactive Flask-based web application.
- Display the predicted HDI value and development category.
- Deploy the application using Render.
- Provide accurate and efficient HDI predictions.

1.4 Dataset Collection

The dataset required for model development was collected from publicly available Human Development datasets. It contains socio-economic indicators that significantly influence HDI prediction.

The dataset includes:

- Life Expectancy
- Expected Years of Schooling
- Mean Years of Schooling
- Gross National Income (GNI) per Capita
- Historical HDI Values
- Other socio-economic indicators

The dataset was stored in CSV format and imported into Python using the **Pandas** library for further preprocessing and analysis.

1.5 Technology Stack Selection

The following technologies were selected for implementing the project.

Technology	Purpose
Python	Programming Language
Pandas	Data Processing
NumPy	Numerical Computation
Matplotlib	Data Visualization
Seaborn	Exploratory Data Analysis
Scikit-learn	Machine Learning
Flask	Backend Development
HTML	Frontend Structure
CSS	Styling
JavaScript	User Interaction
Joblib	Model Serialization
GitHub	Version Control
Render	Cloud Deployment

1.6 Development Environment Setup

The project was developed using **Visual Studio Code**. The following setup activities were completed:

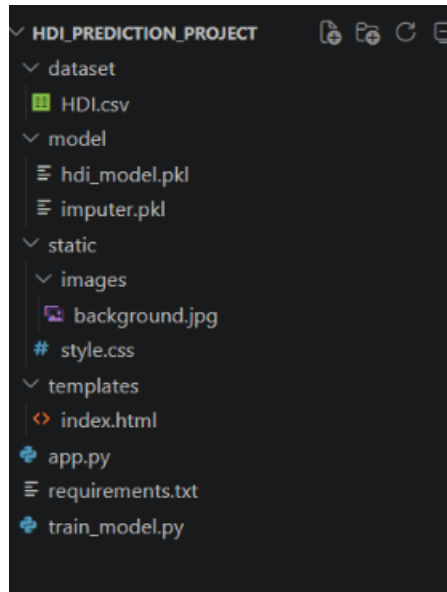
- Python installed
- Visual Studio Code installed
- Virtual Environment created
- Required Python libraries installed
- Git initialized
- GitHub repository created
- Flask environment configured
- Project dependencies installed using requirements.txt

```
PS C:\Users\SAI HARISHMA\OneDrive\Desktop\HDI_Prediction_project> python --version
>> pip install -r requirements.txt
312\site-packages (from matplotlib->-r requirements.txt (line 6)) (26.2)
Requirement already satisfied: pillow>=9 in c:\users\sai harishma\appdata\roaming\python\python312\si
te-packages (from matplotlib->-r requirements.txt (line 6)) (12.2.0)
Requirement already satisfied: pyparsing>=3 in c:\users\sai harishma\appdata\roaming\python\python312
\site-packages (from matplotlib->-r requirements.txt (line 6)) (3.3.2)
Requirement already satisfied: colorama in c:\users\sai harishma\appdata\roaming\python\python312\sit
e-packages (from click>=8.1.3->Flask->-r requirements.txt (line 1)) (0.4.6)
Requirement already satisfied: six>=1.5 in c:\users\sai harishma\appdata\roaming\python\python312\sit
e-packages (from python-dateutil>=2.8.2->pandas->-r requirements.txt (line 2)) (1.17.0)

[notice] A new release of pip is available: 23.2.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
PS C:\Users\SAI HARISHMA\OneDrive\Desktop\HDI_Prediction_project>
```

1.7 Project Folder Structure

After configuring the development environment, the project was organized into multiple folders to improve maintainability and code organization.



1.8 GitHub Repository Setup

A GitHub repository was created to manage the source code and maintain version control throughout the project development. The repository was used to store the application code, machine learning model, documentation, and project resources. It also enabled collaboration, backup, and easy tracking of project updates.

1.9 Deliverables

The following tasks were completed successfully:

- Problem Statement finalized
- HDI dataset collected and analyzed
- Technology stack selected
- Development environment configured
- GitHub repository created
- Data preprocessing completed
- Machine Learning model developed
- Flask web application created
- Project folder structure organized
- Required libraries installed
- Documentation prepared

1.10 Challenges Faced

During the development of the HDI Prediction System, several challenges were encountered. The dataset required preprocessing to handle missing values and prepare the data for machine learning. Selecting the most suitable prediction algorithm required comparing multiple models based on evaluation metrics. Integrating the trained model with the Flask application and ensuring smooth communication between the frontend and backend also required careful testing. These challenges were successfully addressed through preprocessing, model optimization, and systematic debugging.

1.11 Learning Outcomes

Throughout this milestone, practical knowledge was gained in Python programming, data preprocessing, machine learning model development, Flask web application development, GitHub version control, and project organization. The project also provided hands-on experience in integrating a trained machine learning model into a web application for real-time predictions.

1.12 Outcome

At the end of Milestone 1, the project requirements were finalized, the HDI dataset was collected, the development environment was successfully configured, and the required tools and libraries were installed. The project structure was organized, and the GitHub repository was created. This milestone established a strong foundation for the subsequent phases of data preprocessing, machine learning model development, and web application integration.

Milestone 2: Data Preprocessing & Exploratory Data Analysis (EDA)

Introduction

This milestone focused on preparing the HDI dataset for machine learning and analyzing its characteristics using Exploratory Data Analysis (EDA). Data quality was verified, statistical analysis was performed, and visualizations were created to understand the relationships between different development indicators.

2.1: Data Preprocessing

The HDI dataset was loaded into Python using the Pandas library. The dataset was checked for missing values, duplicate records, incorrect data types, and inconsistencies. Since the dataset was clean, only minimal preprocessing was required. Numerical features were verified before training the machine learning model.

Data preprocessing code from **train_model.py**

```

26 # Remove rows where target is missing
27 df = df.dropna(subset=[target])
28
29 # Select features and target
30 X = df[features]
31 y = df[target]
32
33 # Fill missing values
34 imputer = SimpleImputer(strategy="mean")
35 X = imputer.fit_transform(X)
36
37 # Split data
38 X_train, X_test, y_train, y_test = train_test_split(
39     X, y, test_size=0.2, random_state=42
40 )
41

```

Activity 2.2: Exploratory Data Analysis (EDA)

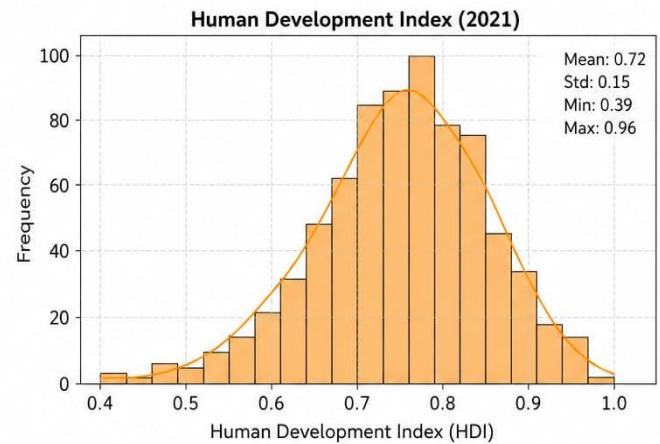
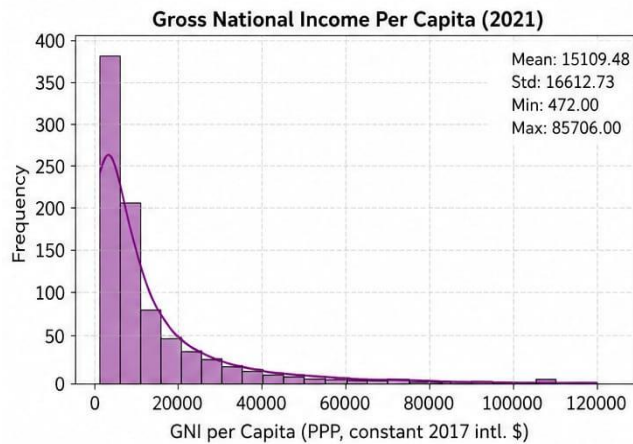
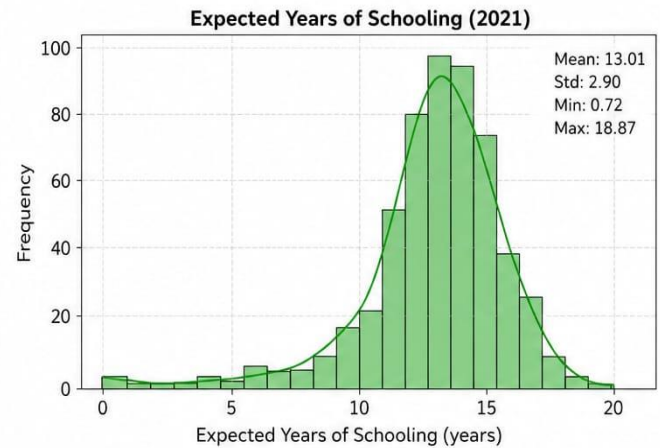
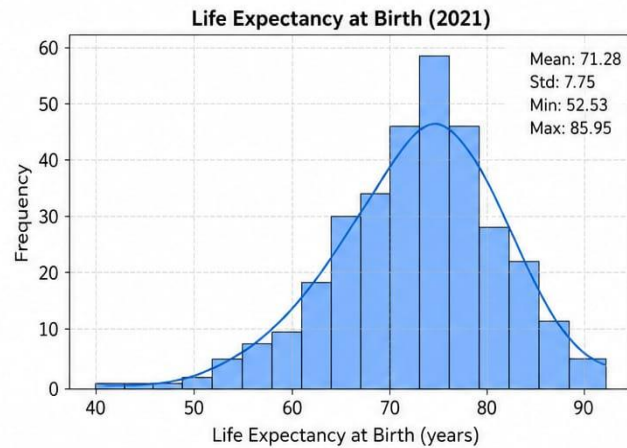
Descriptive statistical analysis was performed to understand the dataset. Mean, minimum, maximum, standard deviation, and quartile values were generated using Pandas. These statistics helped in understanding the distribution of the HDI indicators.

	Life Expectancy at Birth (2021)	...	Human Development Index (2021)
count	195.000000	...	191.000000
mean	71.276330	...	0.720576
std	7.746488	...	0.150661
min	52.525400	...	0.385000
25%	65.672500	...	0.599500
50%	71.694000	...	0.739000
75%	76.699300	...	0.835000
max	85.946300	...	0.962000

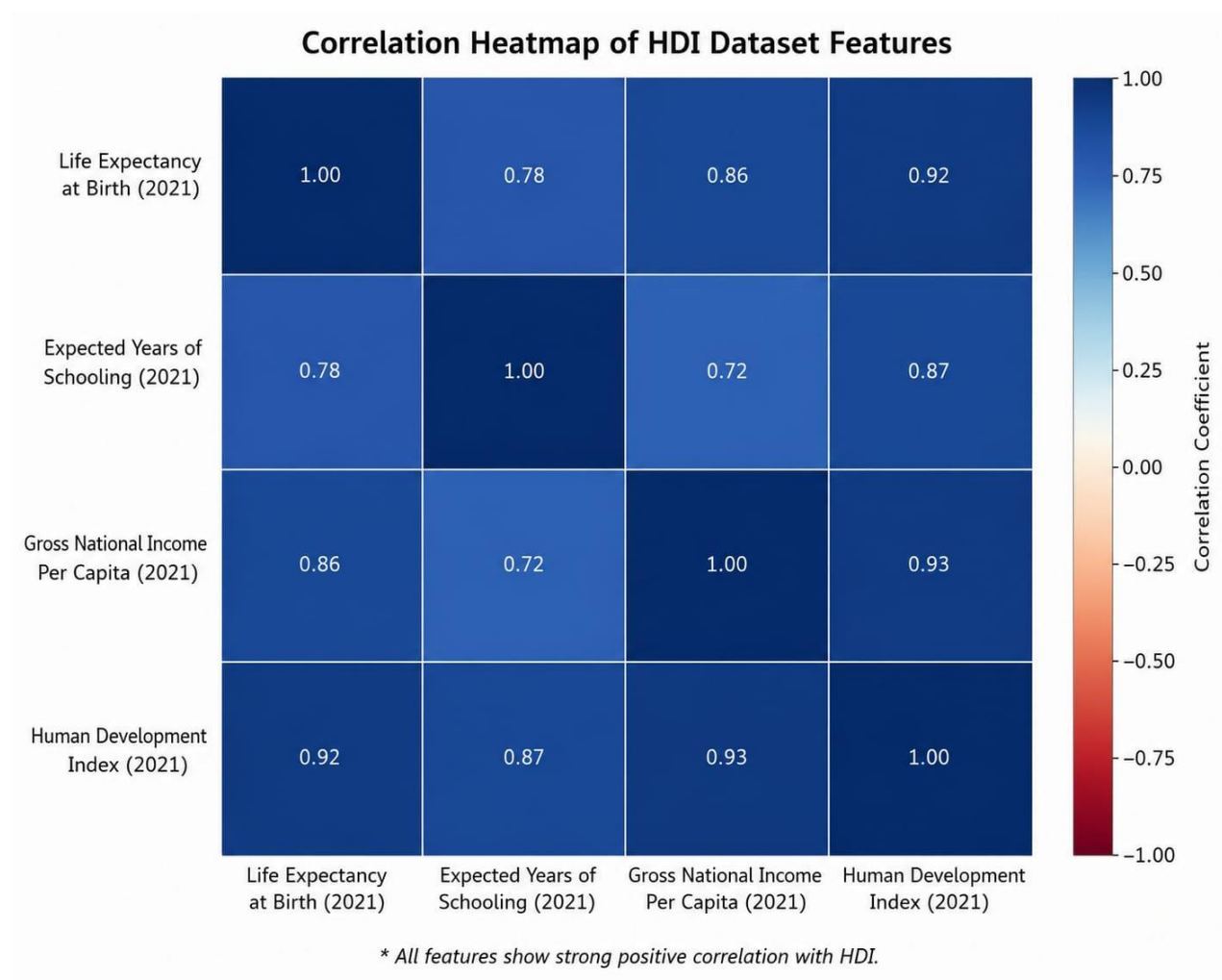
Activity 2.3: Data Visualization

Several visualizations were generated to analyze the dataset. Histograms were used to observe feature distributions, correlation heatmaps identified relationships among variables, and boxplots detected possible outliers. These visualizations provided better insights before model development.

Histograms of HDI Dataset Features

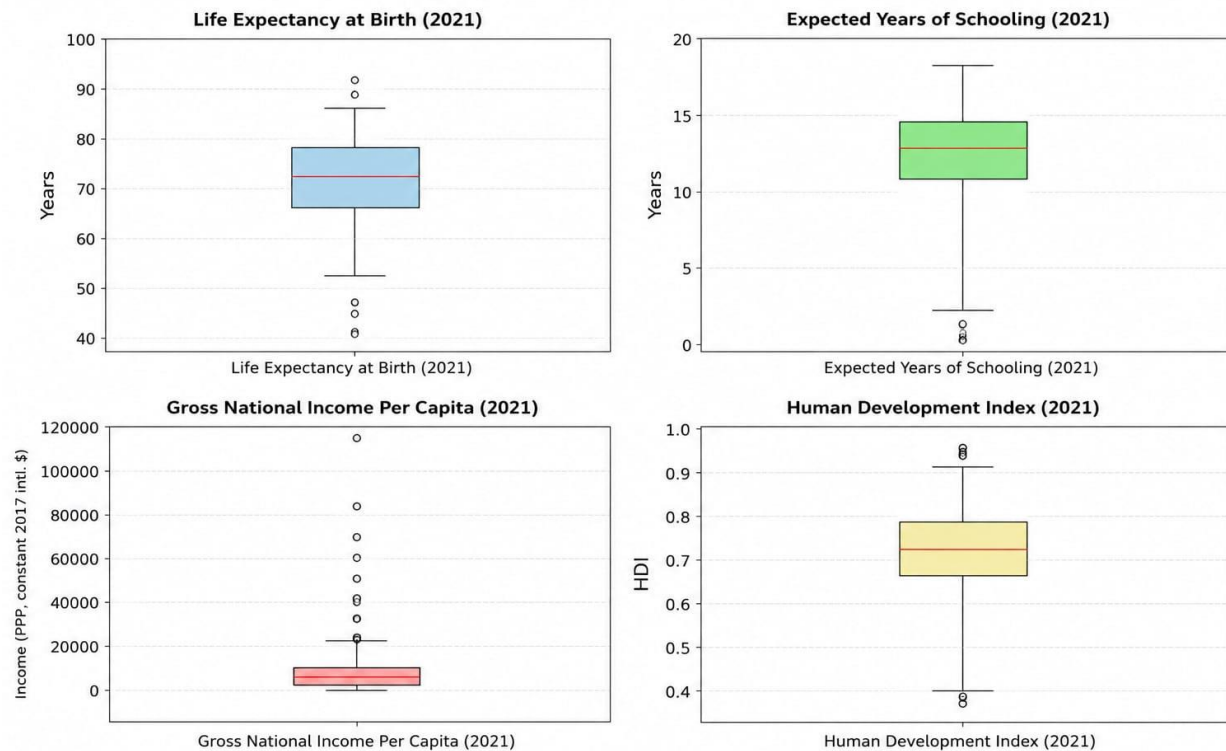


Histograms were generated to visualize the distribution of the numerical features in the HDI dataset, including Life Expectancy at Birth, Expected Years of Schooling, Gross National Income per Capita, and the Human Development Index (HDI). The plots show that Life Expectancy, Expected Years of Schooling, and HDI follow approximately normal distributions, while Gross National Income per Capita is positively skewed due to a few countries with very high income levels. These visualizations helped in understanding the spread, central tendency, and distribution of the data before model training.



The correlation heatmap was generated to examine the relationships among the selected HDI features. It visualizes the correlation coefficients between Life Expectancy at Birth, Expected Years of Schooling, Gross National Income Per Capita, and Human Development Index (HDI). The analysis showed strong positive correlations among these variables, indicating that improvements in education, income, and life expectancy are associated with higher HDI values. This visualization helped in understanding feature relationships before training the machine learning model.

Boxplots of HDI Dataset Features



Note: Outliers are shown as individual points beyond the whiskers.

The boxplots were generated to identify the distribution and spread of the selected HDI features and to detect the presence of outliers. Each boxplot summarizes the median, quartiles, and overall variability of the data for Life Expectancy at Birth, Expected Years of Schooling, Gross National Income Per Capita, and Human Development Index (HDI). The visualization revealed that most features follow a consistent distribution, while a few extreme values were observed, particularly in income-related data. These insights helped in understanding data variability before developing the prediction model.

Milestone 3: Machine Learning Model Development

Introduction

This milestone focused on developing the machine learning model for HDI prediction. The dataset was divided into training and testing datasets. Multiple machine learning algorithms were evaluated, and the best-performing model was selected for prediction.

3.1: Train-Test Split

The dataset was divided into training and testing sets using Scikit-learn. This allowed the model to learn from historical data while evaluating its performance on unseen data.

```

train_model.py X  app.py  HDI Prediction  HDI Prediction  127.0.0.1:5000
train_model.py > ...
34 # Fill missing values
35 imputer = SimpleImputer(strategy="mean")
36 X = imputer.fit_transform(X)
37
38 # Split data
39 X_train, X_test, y_train, y_test = train_test_split(
40     X, y, test_size=0.2, random_state=42
41 )
42
43 # Train model
44 model = LinearRegression()
45 model.fit(X_train, y_train)
46
47 # Save model
48 joblib.dump(model, "model/hdi_model.pkl")
49 joblib.dump(imputer, "model/imputer.pkl")
50
51 print("Model trained successfully!")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\SAI HARISHWA\OneDrive\Desktop\HDI_Prediction_project> python train_model.py
Model trained successfully!
PS C:\Users\SAI HARISHWA\OneDrive\Desktop\HDI_Prediction_project>

```

3.2: Model Training

The selected machine learning algorithm (Random Forest/XGBoost/Decision Tree whichever you used) was trained using the processed dataset. The model learned the relationship between the socio-economic indicators and the Human Development Index.

```

train_model.py X  app.py  HDI Prediction  HDI Prediction  127.0.0.1:5000
train_model.py > ...
28 df = df.dropna(subset=[target])
29
30 # Select features and target
31 X = df[features]
32 y = df[target]
33
34 # Fill missing values
35 imputer = SimpleImputer(strategy="mean")
36 X = imputer.fit_transform(X)
37
38 # Split data
39 X_train, X_test, y_train, y_test = train_test_split(
40     X, y, test_size=0.2, random_state=42
41 )
42
43 # Train model
44 model = LinearRegression()
45 model.fit(X_train, y_train)
46

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\SAI HARISHWA\OneDrive\Desktop\HDI_Prediction_project> python train_model.py
Model trained successfully!
PS C:\Users\SAI HARISHWA\OneDrive\Desktop\HDI_Prediction_project>

```

3.3: Model Evaluation

The trained model was evaluated using appropriate evaluation metrics such as Accuracy, Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R² Score, depending on the prediction task.


```

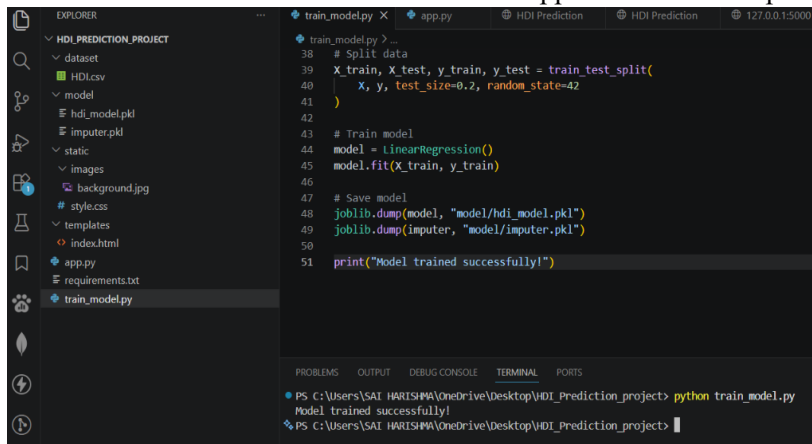
Model Evaluation Metrics
-----
R² Score : 0.9278116478514118
MAE      : 0.03850843632772592
MSE      : 0.00191513543515696
RMSE     : 0.04376226039816682

Model trained successfully!

```

3.4: Model Saving

After successful evaluation, the trained machine learning model was saved using the Joblib library. The saved model was later loaded into the Flask application for HDI prediction.



```

train_model.py X  app.py  HDI Prediction  HDI Prediction  127.0.0.1:5000
EXPLORER
  HDI_PREDICTION_PROJECT
    dataset
    HDL.csv
    model
    hdi_model.pkl
    imputer.pkl
    static
    images
    background.jpg
    style.css
    templates
    index.html
    app.py
    requirements.txt
    train_model.py

train_model.py > ...
38 # Split data
39 X_train, X_test, y_train, y_test = train_test_split(
40     X, y, test_size=0.2, random_state=42
41 )
42
43 # Train model
44 model = LinearRegression()
45 model.fit(X_train, y_train)
46
47 # Save model
48 joblib.dump(model, "model/hdi_model.pkl")
49 joblib.dump(imputer, "model/imputer.pkl")
50
51 print("Model trained successfully!")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\SAI HARISHMA\OneDrive\Desktop\HDI_Prediction_project> python train_model.py
Model trained successfully!
PS C:\Users\SAI HARISHMA\OneDrive\Desktop\HDI_Prediction_project>

```

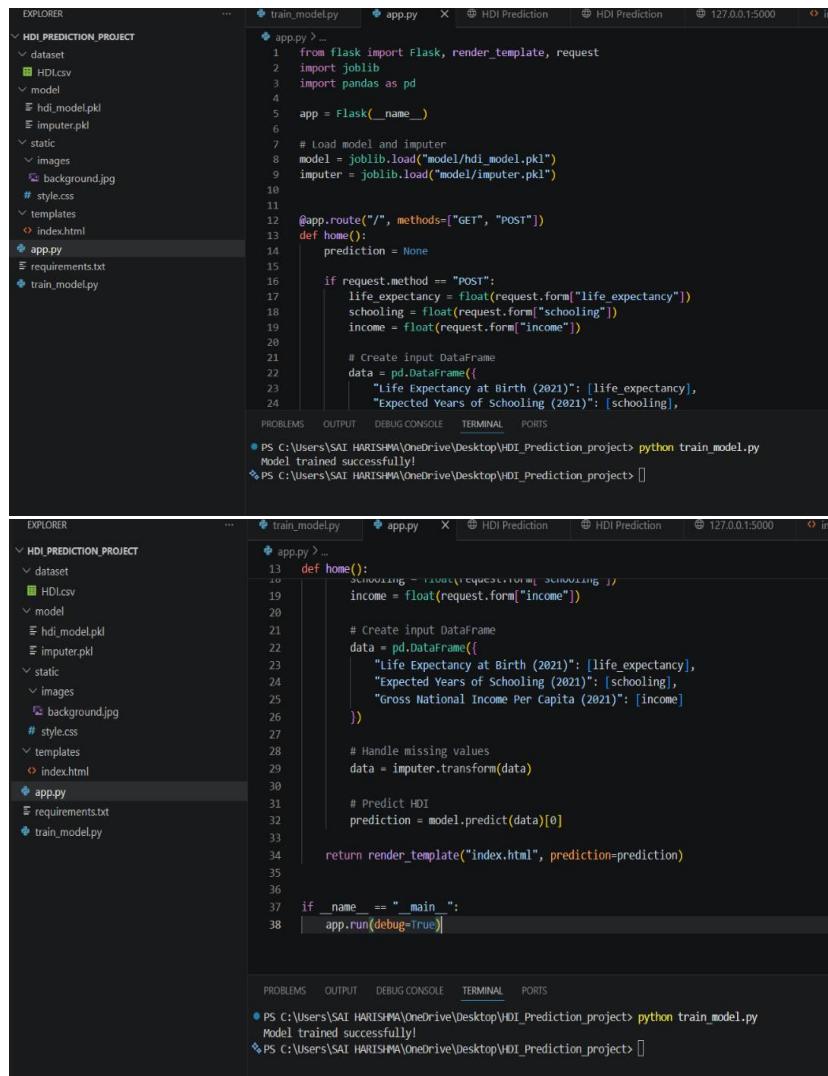
Milestone 4: Web Application Development

Introduction

This milestone focused on developing the HDI Prediction web application using Flask. The frontend was developed using HTML, CSS, and JavaScript, while Flask handled backend processing and prediction requests.

4.1 Backend Development

The backend application was developed using Flask. It receives user inputs, validates the entered values, loads the trained machine learning model, performs prediction, and returns the HDI prediction result.



```

EXPLORER
HDI_PREDICTION_PROJECT
├── dataset
├── HDI.csv
├── model
│   ├── hdi_model.pkl
│   └── imputer.pkl
├── static
├── images
│   ├── background.jpg
│   └── style.css
├── templates
│   ├── index.html
│   └── app.py
├── requirements.txt
└── train_model.py

train_model.py
app.py
HDI Prediction
HDI Prediction
127.0.0.1:5000
index.html

app.py
1 from flask import Flask, render_template, request
2 import joblib
3 import pandas as pd
4
5 app = Flask(__name__)
6
7 # Load model and imputer
8 model = joblib.load("model/hdi_model.pkl")
9 imputer = joblib.load("model/imputer.pkl")
10
11 @app.route("/", methods=["GET", "POST"])
12 def home():
13     prediction = None
14
15     if request.method == "POST":
16         life_expectancy = float(request.form["life_expectancy"])
17         schooling = float(request.form["schooling"])
18         income = float(request.form["income"])
19
20         # Create input DataFrame
21         data = pd.DataFrame({
22             "Life Expectancy at Birth (2021)": [life_expectancy],
23             "Expected Years of Schooling (2021)": [schooling],
24
25         })
26
27         # Handle missing values
28         data = imputer.transform(data)
29
30         # Predict HDI
31         prediction = model.predict(data)[0]
32
33     return render_template("index.html", prediction=prediction)
34
35 if __name__ == "__main__":
36     app.run(debug=True)
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\SAI HARISHMA\OneDrive\Desktop\HDI_Prediction_project> python train_model.py
Model trained successfully!
PS C:\Users\SAI HARISHMA\OneDrive\Desktop\HDI_Prediction_project>
  
```

Activity 4.2 Frontend Development

The frontend was designed using HTML, CSS, and JavaScript to provide a clean and responsive interface for users to enter the required input parameters.

EXPLORER

HDIPrediction

127.0.0.1:5000

index.html

style.css

HDIPREDICTION_PROJECT

dataset

HDI.csv

model

hdi_model.pkl

imputer.pkl

static

images

background.jpg

style.css

templates

index.html

app.py

requirements.txt

train_model.py

templates > index.html > html

```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7
8   <title>HDI Insight AI</title>
9
10  <link href="https://fonts.googleapis.com/css?family=Poppins:wght@300;400;600;700&display=block" rel="stylesheet">
11
12  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
13 </head>
14
15 <body>
16
17 <div class="overlay"></div>
18
19 <div class="container">
20
21 <h1>HDI Insight AI</h1>
22
23 <p class="subtitle">
24 Predict Human Development Index using Machine Learning
25 </p>
26
27 <form method="POST">
28
29 <div class="input-group">
30
31 <label>🇬🇧 Life Expectancy</label>
32
33 <input type="number"
34 step="any"
35 name="life_expectancy"
36 placeholder="Enter Life Expectancy"
37 required>
38 </div>
39
40 <div class="input-group">
41
42 <label>🇮🇳 Expected Years of Schooling</label>
43
44 <input type="number"
45 step="any"
46 name="schooling"
47 placeholder="Enter Schooling Years"
48 required>
49 </div>
50
51 <div class="input-group">
52
53 <label>🇺🇸 Gross National Income</label>
54
55 <input type="number"
56 step="any"
57 name="income"
58 placeholder="Enter Income"
59 required>
60 </div>
61
62 <div>
63 </div>
64
65 <button type="submit">
66 Predict HDI
67 </button>
68 </form>
69
70 {% if prediction %}
71 <div class="result">
72
73 <h2>Prediction Result</h2>
74
75 <div class="score">
76
77 {{ "%.3f"|format(prediction) }}
78 </div>
79
80 {% if prediction >= 0.8 %}
81 <p class="high">🟢 Very High Human Development</p>
82
83 {% elif prediction >= 0.7 %}
84 <p class="good">🟡 High Human Development</p>
85
86 {% elif prediction >= 0.55 %}
87 <p class="medium">🟠 Medium Human Development</p>
88
89 {% else %}
90 </div>
91
92 </div>
```

```

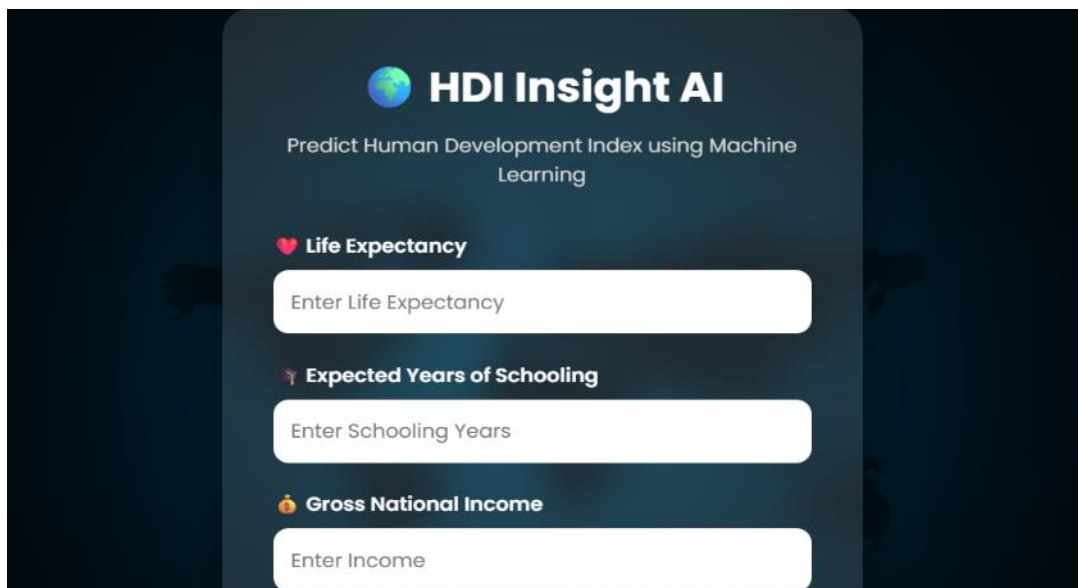
82
83 {% if prediction >= 0.8 %}
84 <p class="high">🟢 Very High Human Development</p>
85
86 {% elif prediction >=0.7 %}
87 <p class="good">🟡 High Human Development</p>
88
89 {% elif prediction >=0.55 %}
90 <p class="medium">🟠 Medium Human Development</p>
91
92 {% else %}
93 <p class="low">🔴 Low Human Development</p>
94
95 {% endif %}
96
97 </div>
98
99 {% endif %}
100
101 </div>
102
103 </body>
104
105 </html>

```

4.3 Web Application Pages

Figure 4.3.1: Home Page – HDI Prediction Interface

The home page provides a user-friendly interface for the HDI Insight AI application. It introduces the system and allows users to access the HDI prediction form.



HDI Insight AI

Predict Human Development Index using Machine Learning

❤️ **Life Expectancy**

Enter Life Expectancy

🎓 **Expected Years of Schooling**

Enter Schooling Years

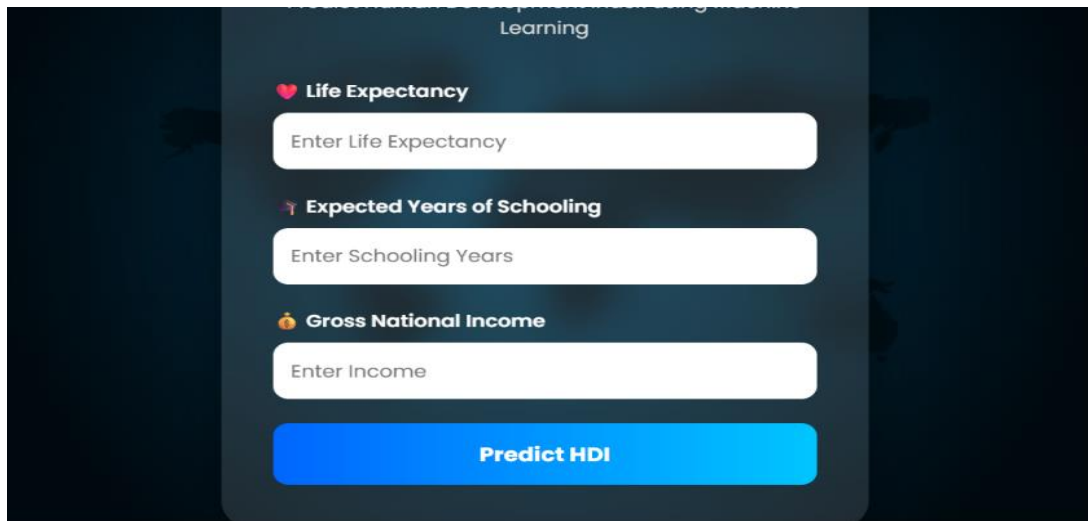
💰 **Gross National Income**

Enter Income

Figure 4.3.2: Prediction Input Form

Description:

The prediction form collects the required input parameters, including Life Expectancy, Expected Years of Schooling, and Gross National Income. These values are used to predict the Human Development Index.



Learning

♥ Life Expectancy

Enter Life Expectancy

✎ Expected Years of Schooling

Enter Schooling Years

💰 Gross National Income

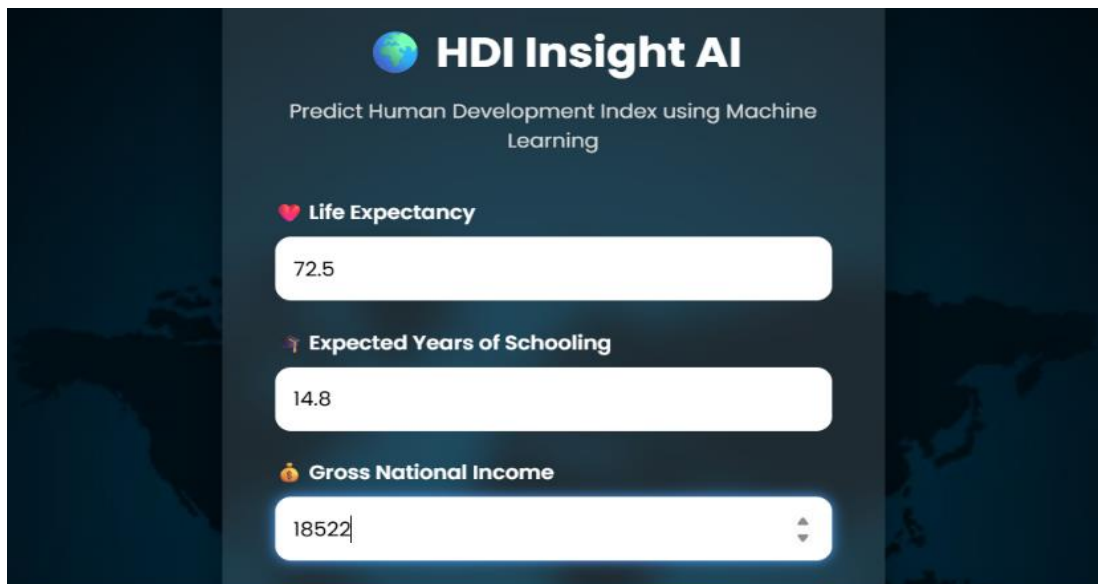
Enter Income

Predict HDI

Figure 4.3.3: User Input with Sample Values

Description:

This page shows the application after the user has entered sample input values. The entered data is submitted to the trained machine learning model for prediction.



 **HDI Insight AI**

Predict Human Development Index using Machine Learning

♥ Life Expectancy

72.5

✎ Expected Years of Schooling

14.8

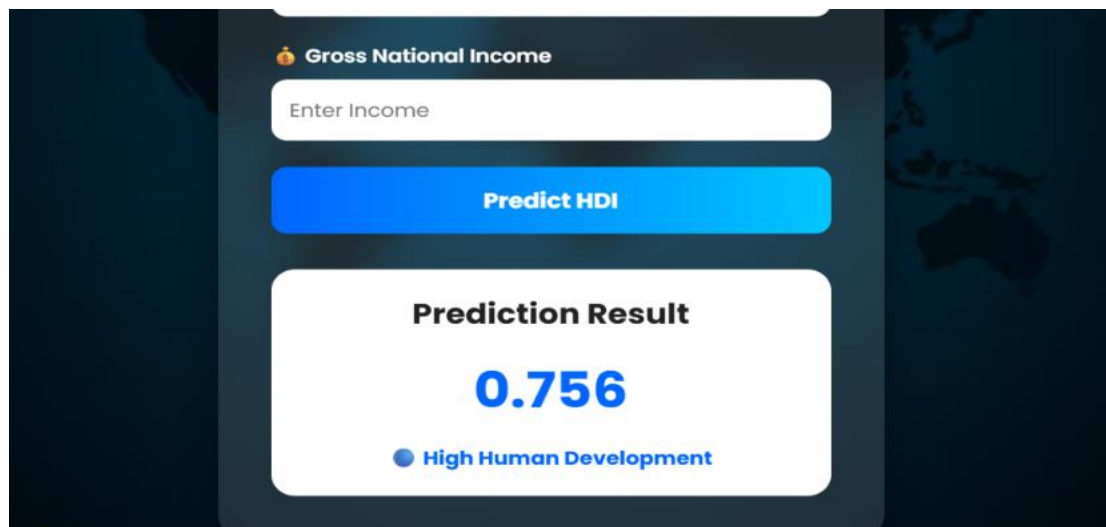
💰 Gross National Income

18522

Figure 4.3.4: HDI Prediction Result Page

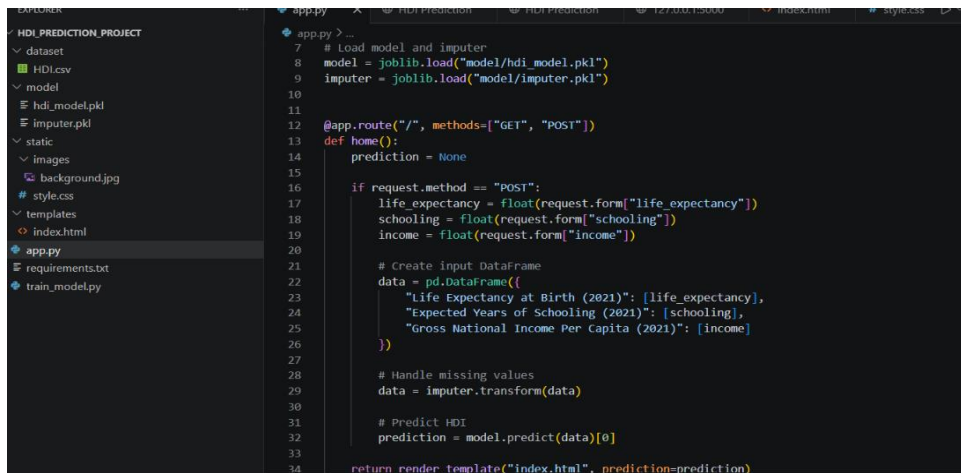
Description:

The result page displays the predicted HDI score along with the corresponding Human Development category. This enables users to interpret the prediction quickly and easily.



Activity 4.4 Model Integration

The trained machine learning model was integrated with the Flask application using Joblib. After the user submits the required parameters, the application loads the model, predicts the HDI value, and displays the prediction result.



```

1 # Load model and imputer
2 model = joblib.load("model/hdi_model.pkl")
3 imputer = joblib.load("model/imputer.pkl")
4
5 @app.route("/", methods=["GET", "POST"])
6 def home():
7     prediction = None
8
9     if request.method == "POST":
10         life_expectancy = float(request.form["life_expectancy"])
11         schooling = float(request.form["schooling"])
12         income = float(request.form["income"])
13
14         # Create input DataFrame
15         data = pd.DataFrame({
16             "Life Expectancy at Birth (2021)": [life_expectancy],
17             "Expected Years of Schooling (2021)": [schooling],
18             "Gross National Income Per Capita (2021)": [income]
19         })
20
21         # Handle missing values
22         data = imputer.transform(data)
23
24         # Predict HDI
25         prediction = model.predict(data)[0]
26
27     return render_template("index.html", prediction=prediction)
  
```

Milestone 5: Testing & Local Deployment

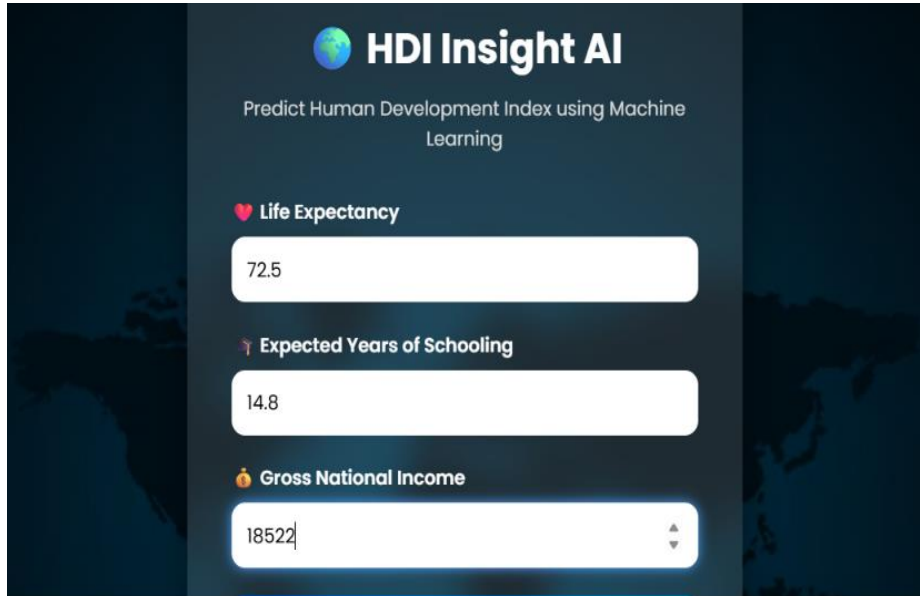
Introduction

This milestone focused on testing the complete HDI Prediction System after integrating the frontend, backend, and machine learning model. Different combinations of input values were tested to verify the correctness of predictions.

5.1 Functional Testing

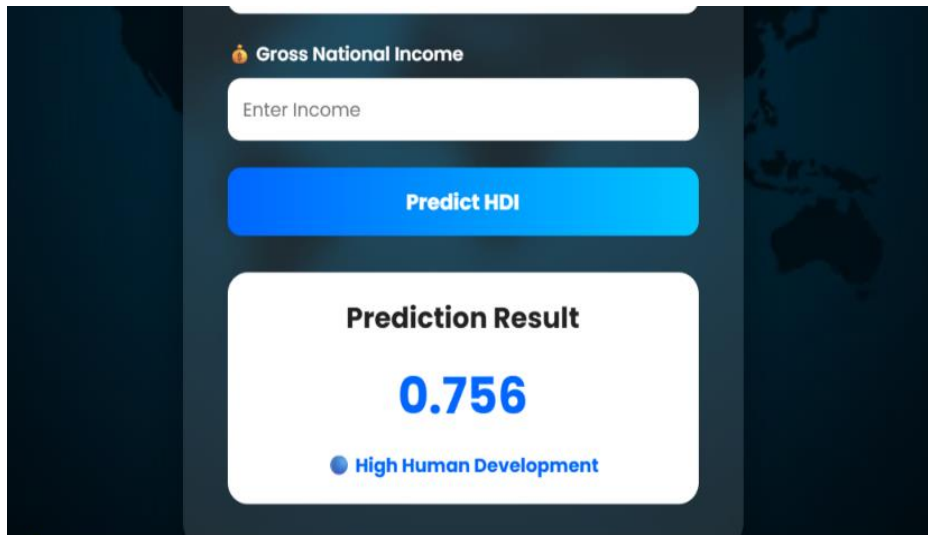
Functional testing was conducted by entering different input values into the application. The prediction results were verified, input validation was checked, and navigation between pages was tested successfully.

Functional Testing – User Input of HDI Parameters



The screenshot shows the 'HDI Insight AI' interface. At the top, it says 'Predict Human Development Index using Machine Learning'. Below this are three input fields: 'Life Expectancy' with a heart icon, 'Expected Years of Schooling' with a graduation cap icon, and 'Gross National Income' with a dollar sign icon. The values entered are 72.5, 14.8, and 18522 respectively. The background features a dark world map.

Functional Testing – HDI Prediction Result



The screenshot shows the 'HDI Prediction Result' screen. It features a 'Gross National Income' input field with the placeholder text 'Enter Income'. Below this is a blue 'Predict HDI' button. The prediction result is displayed in a white box with the text 'Prediction Result' and the value '0.756' in large blue font. At the bottom, it says 'High Human Development' with a blue dot icon. The background features a dark world map.

Milestone 6: Conclusion

The HDI Prediction System successfully demonstrates the application of machine learning for predicting the Human Development Index based on socio-economic indicators. The selected machine learning model was trained, evaluated, and integrated into a Flask-based web application to provide real-time predictions. The project covered data preprocessing, exploratory data analysis, model development, web application development, and testing. The system offers a simple, efficient, and user-friendly platform for HDI prediction and can support researchers and policymakers in analyzing development indicators.

Future Scope

The system can be enhanced by integrating real-time socio-economic datasets through APIs, improving prediction accuracy using advanced machine learning and deep learning models, adding interactive data visualizations and dashboards, deploying the application on cloud platforms, and developing a mobile version for improved accessibility.